

Security Fundamentals and Low-Level Programming

Memory, Machine Code, and the Attack Surface

Bernard

May 25, 2026

“You don’t understand security until you understand what the processor actually does.”

Contents

I	Foundations	7
1	Introduction to Low-Level Security	9
1.1	Why Low-Level Security Matters	9
1.2	Security vs. Safety vs. Correctness	10
1.3	The CIA Triad	10
1.4	A Brief History of Low-Level Security	10
1.5	Who This Book Is For	11
1.6	How This Book Is Organised	11
1.7	Exercises	11
2	The Memory Model	13
2.1	Virtual Memory	13
2.2	Paging	14
2.3	Memory Layout of a Process	14
2.3.1	The Stack	15
2.3.2	The Heap	15
2.4	Memory Protection and the NX Bit	15
2.5	TLB and Caching	16
2.6	Security Implications of the Memory Model	16
2.7	Exercises	16
3	Processor Architecture and Machine Code	17
3.1	The x86-64 Register File	17
3.1.1	Sub-Register Access	18
3.2	Basic Instructions	18
3.2.1	Data Movement	18
4	Memory Unsafety	19
4.1	What Is Memory Unsafety?	19
4.2	Classification of Memory Unsafety	19
4.3	Stack Buffer Overflows	20
4.3.1	Functions That Enable Overflows	20
4.4	Heap Overflows	21
4.5	Use-After-Free	21
4.5.1	Double Free	22
4.6	Format String Vulnerabilities	22
4.7	Integer Overflows Leading to Memory Unsafety	23
4.8	The Eternal War	23
4.9	Exercises	23

II	Exploitation and Defence	25
5	Exploitation Techniques	27
5.1	From Bug to Exploit	27
5.2	Shellcode	27
5.2.1	A Minimal Shellcode: <code>execve("/bin/sh")</code>	28
5.2.2	Null-Free Constraint	28
5.3	Classic Stack Smashing	28
5.3.1	NOP Sleds	29
5.4	Return-to-libc (<code>ret2libc</code>)	29
5.5	Return-Oriented Programming (ROP)	29
5.5.1	Finding Gadgets	30
5.5.2	Example: ROP <code>execve</code>	30
5.6	Format String Exploitation	30
5.6.1	Memory Leak	30
5.6.2	Arbitrary Write with <code>%n</code>	31
5.7	Heap Exploitation	31
5.8	The Exploitation Pipeline	31
5.9	Exercises	32
6	Protections and Mitigations	33
6.1	The Mitigation Stack	33
6.2	Non-Executable Memory (NX/DEP)	34
6.3	Address Space Layout Randomisation (ASLR)	34
6.3.1	ASLR Effectiveness	34
6.3.2	Bypassing ASLR	34
6.3.3	Position-Independent Executables (PIE)	35
6.4	Stack Canaries	35
6.4.1	Bypassing Stack Canaries	35
6.5	Control-Flow Integrity (CFI)	36
6.5.1	Forward-Edge CFI	36
6.5.2	Backward-Edge CFI (Shadow Stacks)	36
6.6	Compiler Hardening	36
6.7	Runtime Defences	37
6.7.1	<code>seccomp</code>	37
6.7.2	Sanitizers	37
6.8	The Arms Race	37
6.9	Exercises	38
7	Kernel Security	39
7.1	Privilege Rings	39
7.2	The Kernel Attack Surface	39
7.2.1	System Calls	40
7.2.2	Device Drivers	40
7.2.3	<code>procfs</code> and <code>sysfs</code>	40
7.2.4	<code>ioctl</code>	40
7.3	Kernel Vulnerability Classes	40
7.3.1	Kernel Stack Overflow	40
7.3.2	Kernel Heap Overflow	41
7.3.3	Use-After-Free in Kernel	41
7.3.4	Double Fetch	41
7.4	Kernel Exploitation Primitives	41

7.4.1	commit_creds Exploit	42
7.4.2	Modprobe_Path Technique	42
7.5	Kernel Defences	42
7.5.1	KASLR	42
7.5.2	SMEP and SMAP	42
7.5.3	KPTI	42
7.5.4	Kernel CFI	43
7.5.5	seccomp for the Kernel	43
7.6	The Kernel Exploit Pipeline	43
7.7	Exercises	43
III	Applied Low-Level Security	45
8	Network Security from the Low-Level Perspective	47
8.1	Network Packets at the Byte Level	47
8.2	Protocol Parsing Vulnerabilities	48
8.2.1	Length Field Validation	48
8.2.2	TTL and Fragment Handling	48
8.3	Buffer Overflow in Network Parsers	48
8.4	Socket-Level Attacks	49
8.4.1	TCP Desynchronisation	49
8.4.2	TCP Side Channels	49
8.5	Kernel-Bypass Networking	49
8.5.1	Security Implications	49
8.5.2	eXpress Data Path (XDP)	50
8.6	Wireless and Low-Level Attacks	50
8.7	Exercises	50
9	Cryptography for Systems Programmers	51
9.1	Cryptographic Primitives for Systems Programming	51
9.1.1	Symmetric Encryption	51
9.1.2	Asymmetric (Public-Key) Encryption	52
9.1.3	Hash Functions	52
9.2	Common Low-Level Crypto Pitfalls	52
9.2.1	Constant-Time Programming	52
9.2.2	Random Number Generation	53
9.2.3	Padding Oracle Attacks	53
9.3	Memory Safety in Cryptographic Code	54
9.3.1	Secret Management	54
9.3.2	Library-Level Vulnerabilities	54
9.4	Authenticated Encryption	54
9.5	Exercises	55
10	Secure Development Practices	57
10.1	The Secure Development Lifecycle (SDL)	57
10.2	Threat Modelling for Low-Level Systems	57
10.2.1	Identifying Trust Boundaries	58
10.2.2	The STRIDE Model	58
10.3	Coding Standards for Memory Safety	58
10.3.1	Rule 1: Always Bounds-Check	58
10.3.2	Rule 2: Use Safe String Functions	59

10.3.3 Rule 3: Validate All Integer Arithmetic	59
10.3.4 Rule 4: Avoid Variable-Length Arrays (VLAs)	59
10.3.5 Rule 5: Initialize All Memory	59
10.4 Fuzzing for Security Bugs	60
10.5 Static Analysis	60
10.6 Compiler Hardening for Release Builds	61
10.7 Code Review Checklist for Low-Level Security	61
10.8 Building a Security Culture	61
10.9 Exercises	62

Part I

Foundations

Chapter 1

Introduction to Low-Level Security

“You don’t understand security until you understand what the processor actually does.”

Learning Goals

- Define security in the context of low-level systems and motivate the study of computer architecture.
- Distinguish between security, safety, and correctness.
- Identify the three core principles of the CIA triad and their relevance to systems programming.
- Trace the historical roots of low-level security vulnerabilities.
- Outline the roadmap of this book.

1.1 Why Low-Level Security Matters

Most security discussions focus on high-level concerns—passwords, firewalls, encryption protocols, web application vulnerabilities. These are important, but they rest on a foundation that is often ignored: the behaviour of the processor, the memory subsystem, and the machine code that runs on them.

A web application firewall cannot help if an attacker has already achieved arbitrary code execution through a buffer overflow. A strong password is irrelevant if a format string vulnerability leaks the master password from memory. Encryption is useless if the plaintext lingers on the heap after use.

Low-level security is the study of what happens at the instruction-set architecture (ISA) level, in the memory model exposed by the operating system, and in the interface between user code and the kernel. It asks: given that the processor executes exactly the sequence of bytes we provide, what can go wrong?

Definition 1.1 (Low-Level Security). **Low-level security** is the branch of computer security concerned with vulnerabilities and defences that arise from the behaviour of processors, memory subsystems, operating system kernels, and the machine-code interface between software and hardware.

Example 1.1 (The Real Cost of Low-Level Bugs). In 2014, the Heartbleed vulnerability (CVE-2014-0160) allowed attackers to read arbitrary memory from OpenSSL servers. The bug was a missing bounds check in a single `memcpy` call. It exposed passwords, private keys, and session

data from millions of servers. No cryptographic weakness was involved—the bug was purely a low-level memory safety issue.

1.2 Security vs. Safety vs. Correctness

A useful distinction before we proceed:

- **Correctness:** the program behaves according to its specification. A correct program produces the right output for every legal input.
- **Safety:** the program does not cause harm, even when it behaves incorrectly. A type-safe language prevents memory corruption even if the logic is buggy.
- **Security:** the program resists intentional subversion by an adversary. A secure program must be correct and safe *under attack*.

These are nested requirements. Security subsumes safety, which subsumes correctness—but only in the direction of the adversary’s goals. Many correct programs are unsafe (they corrupt memory on edge cases). Many safe programs are insecure (they leak information through side channels).

Principle 1.1 (The Attacker’s Advantage). An attacker needs only one bug. The defender must eliminate all bugs. This asymmetry drives the entire field of low-level security.

1.3 The CIA Triad

Security is traditionally organised around three goals:

Confidentiality Preventing unauthorised information disclosure. Memory disclosure vulnerabilities (Heartbleed, Spectre) violate confidentiality.

Integrity Preventing unauthorised modification. Buffer overflows that overwrite return addresses violate integrity.

Availability Ensuring the system remains usable. Denial-of-service attacks violate availability.

In low-level security, confidentiality and integrity are the primary concerns. An attacker who achieves arbitrary code execution has breached both: they can read any memory (confidentiality) and write any memory (integrity).

1.4 A Brief History of Low-Level Security

The field has evolved through several eras:

Era 1: The Stack Smash (1988–2000) The Morris worm (1988) exploited a buffer overflow in `fingerd`. Aleph One’s seminal paper “Smashing the Stack for Fun and Profit” [1] in 1996 codified the technique. For years, stack buffer overflows dominated the vulnerability landscape.

Era 2: Heap and Format String (2000–2005) Attackers moved beyond the stack. Heap overflows, use-after-free bugs, and format string vulnerabilities [2] expanded the attack surface. The rise of network services created a pipeline: network input → parsing → memory corruption → code execution.

Era 3: Mitigation Arms Race (2005–2015) Defenders introduced stack canaries, non-executable stacks (DEP), address space layout randomisation (ASLR), and control-flow integrity [3]. Attackers responded with return-oriented programming (ROP) [4] and information leaks.

Era 4: Beyond Memory Corruption (2015–present) Side-channel attacks (Spectre, Meltdown), kernel exploitation techniques, and the rise of memory-safe languages (Rust, Go) characterise the current era. The fundamental problems remain unsolved: C and C++ code still dominate systems programming, and every new memory corruption bug is a potential exploit.

1.5 Who This Book Is For

This book is for programmers who want to understand security from the silicon up. You should know C at an intermediate level—pointers, manual memory management, and basic data structures. You do not need prior security experience. Assembly language will be introduced as needed.

1.6 How This Book Is Organised

The book is divided into three parts:

Part I: Foundations (Chapters 1–4) covers the essential background: how memory works, how processors execute code, and what memory unsafety means at the machine level. By the end of Part I, you will be able to read assembly, understand virtual memory, and identify the root causes of the most common vulnerability classes.

Part II: Exploitation and Defence (Chapters 5–7) covers the techniques attackers use to turn memory corruption into code execution, and the defences that prevent them. You will learn shellcoding, ROP, ASLR bypasses, and the kernel attack surface.

Part III: Applied Low-Level Security (Chapters 8–10) extends the low-level mindset to network protocols, cryptography, and secure development practices.

1.7 Exercises

Exercise 1.1. *Research the Morris worm (1988). What vulnerability did it exploit? How did the security community respond? Write a one-paragraph summary.*

Exercise 1.2. *Identify a recent CVE (2020 or later) classified as a buffer overflow. Read the description and the patch. What was the root cause? How many lines of code changed to fix it?*

Exercise 1.3. *Consider a simple C program that reads a name from stdin and prints a greeting. List three ways an attacker could violate the CIA triad through this program, assuming no security mitigations are enabled.*

Chapter 2

The Memory Model

“All problems in computer science can be solved by another level of indirection, except for the problem of too many levels of indirection.”

David Wheeler

Learning Goals

- Describe the virtual memory abstraction and its components: pages, frames, and page tables.
- Distinguish between stack and heap memory organisation and their lifetime semantics.
- Explain how memory permissions (read, write, execute) are enforced at the hardware level.
- Understand segmentation, paging, and the address translation pipeline.
- Relate memory model features to security properties and vulnerabilities.

2.1 Virtual Memory

Every modern processor provides a *virtual memory* abstraction. Each process sees a private, contiguous address space starting at address 0. The processor and operating system cooperate to map these virtual addresses to physical memory locations.

Definition 2.1 (Virtual Memory). **Virtual memory** is an indirection layer that maps virtual addresses (used by programs) to physical addresses (used by hardware). The mapping is performed by the *memory management unit* (MMU) using data structures called *page tables*.

Why virtual memory? Three reasons:

1. **Isolation:** process A cannot read or write process B’s memory, because A’s virtual addresses map to different physical pages.
2. **Security:** the kernel can mark pages as non-executable, non-writable, or non-readable. Any violation triggers a processor exception.
3. **Convenience:** each process can use a flat address space without coordinating with other processes or worrying about physical memory fragmentation.

Principle 2.1 (The MMU as Security Enforcer). Every memory access goes through the MMU. This means that memory protections are not optional—they are enforced by the processor on *every* load and store instruction, including those in kernel code. A bug in user space cannot bypass the MMU (barring processor vulnerabilities like Meltdown).

2.2 Paging

The dominant virtual memory implementation is *paging*. Virtual and physical memory are divided into fixed-size blocks called *pages* (typically 4 KB on x86-64).

Definition 2.2 (Page, Frame, Page Table). A **page** is a fixed-size block of virtual memory. A **frame** is a fixed-size block of physical memory of the same size. A **page table** is a multi-level data structure that maps virtual pages to physical frames.

x86-64 uses four levels of page tables (PML4, PDPT, PD, PT) for 4 KB pages. A virtual address is split into fields that index each level:

Listing 2.1: Virtual address layout on x86-64 (4KB pages)

```
Virtual address (48 bits):
| PML4 (9) | PDPT (9) | PD (9) | PT (9) | Offset (12) |
```

Each level's entry contains the physical address of the next-level table, plus permission bits:

- P (Present): the page is in physical memory.
- R/W: writable (0 = read-only).
- U/S: user-accessible (0 = supervisor-only).
- NX (No-Execute): code cannot be executed from this page.
- A (Accessed): set by hardware on first access.
- D (Dirty): set by hardware on first write.

Example 2.1 (Address Translation). Consider a load from virtual address 0x7fff00123456. The processor:

1. Extracts bits 47:39 to index PML4 (entry 0x1FF).
2. Reads the PML4 entry, which contains the physical address of the PDPT.
3. Extracts bits 38:30 to index the PDPT.
4. Repeats for PD (bits 29:21) and PT (bits 20:12).
5. Reads the PTE, which contains the physical frame number.
6. Adds the 12-bit page offset (bits 11:0) to form the final physical address.

If any page-table entry has P=0, the processor raises a page fault. If U/S=0 while the processor is in user mode, a protection fault occurs.

2.3 Memory Layout of a Process

A typical process on Linux has the following virtual memory layout:

Listing 2.2: Linux process memory layout (x86-64)

```
High addresses (0xfffffffffff)
+-----+
| Kernel space | Not accessible from user mode
| (mapped in every process) |
+-----+
| Stack | Grows downward (toward lower addresses)
| (per-thread, ~8MB by def) |
+-----+
| Memory mapping region | mmap, shared libraries, heap expansion
| (grows downward) |
+-----+
| Heap | Grows upward (brk/sbrk)
+-----+
| BSS | Uninitialized global data
+-----+
```

```

| Data | Initialized global data
+-----+
| Text (code) | Executable machine code
| (read-only, executable) |
+-----+
Low addresses (0x400000)

```

2.3.1 The Stack

The stack is a contiguous region of memory that grows downward. It stores:

- **Return addresses:** where to resume execution after a function call.
- **Local variables:** automatic storage duration.
- **Saved frame pointers:** the previous base pointer (**rbp**) value.
- **Function arguments:** passed in registers or on the stack.

Definition 2.3 (Stack Frame). A **stack frame** is the region of the stack allocated for a single function invocation. It contains the function’s local variables, saved registers, and the return address. The exact layout is determined by the calling convention (Section ??).

The stack pointer (**rsp**) points to the top of the stack. The base pointer (**rbp**) typically points to the start of the current frame. On x86-64, arguments are passed in registers for the first six integer arguments (**rdi**, **rsi**, **rdx**, **rcx**, **r8**, **r9**); additional arguments go on the stack.

2.3.2 The Heap

The heap is a dynamically allocated memory region. Unlike the stack, heap allocations:

- Have arbitrary lifetimes (must be explicitly freed).
- Can be resized with **realloc**.
- Are managed by an allocator (e.g., **ptmalloc** on Linux).
- Are subject to fragmentation.

Definition 2.4 (Heap Allocator). A **heap allocator** manages the heap region, satisfying **malloc** and **free** requests. It maintains internal metadata (chunk headers, free lists) to track which regions are in use. Corrupting this metadata is a common exploitation technique.

2.4 Memory Protection and the NX Bit

Before the NX (No-Execute) bit was introduced, all memory pages were implicitly executable. This meant that an attacker who could write data to memory (via a buffer overflow) could then jump to that data and execute it as code. The NX bit, introduced in x86-64, allows each page to be independently marked executable or non-executable.

Principle 2.2 (**W^X** (Write XOR Execute)). A memory page should be either writable or executable, but never both. This principle, called **W^X**, prevents the most direct exploitation path: writing shellcode to memory and executing it.

The stack and heap are mapped as readable and writable, but not executable. The code section (**.text**) is mapped as readable and executable, but not writable. An attacker cannot write new code to memory and execute it—they must reuse existing code (Section 5).

2.5 TLB and Caching

Virtual-to-physical address translation would be slow if every memory access required four page-table walks. The *translation lookaside buffer* (TLB) caches recent translations.

Definition 2.5 (TLB). The **TLB** is a hardware cache in the CPU that stores recent virtual-to-physical page mappings. A TLB hit avoids the page-table walk; a TLB miss triggers a hardware walk (or a microcode-assisted walk).

TLBs are small (typically 32–64 entries for L1, up to 2048 for L2). Context switches (switching between processes) flush the TLB, which is why system calls and thread switches have a performance cost.

2.6 Security Implications of the Memory Model

Every feature of the memory model has security implications:

Feature	Security benefit	Security risk
Virtual memory	Process isolation prevents cross-process attacks	Information leaks via shared pages (e.g., KASLR)
Page permissions	NX prevents direct shell-code execution	WX pages in buggy drivers bypass NX
TLB	Fast address translation	TLB side channels can leak address information
Heap allocator	Transparent memory management	Heap metadata corruption leads to arbitrary write
Stack frame layout	Efficient function call mechanism	Return address overwrite hijacks control flow

2.7 Exercises

Exercise 2.1. On *x86-64* with 4 KB pages, how many bytes are used for the virtual address in each of the four page-table fields (PML4, PDPT, PD, PT)? What fraction of the 64-bit address space is currently usable?

Exercise 2.2. Given a virtual address `0x7f0000000000`, compute the indices for PML4, PDPT, PD, and PT. Assume 4 KB pages and 48-bit virtual addresses.

Exercise 2.3. Why do large pages (2 MB or 1 GB) reduce TLB pressure? What is the trade-off for security?

Exercise 2.4. Research the KASLR (Kernel Address Space Layout Randomisation) feature. How does it randomise the kernel’s position in virtual memory? How have attackers bypassed it using TLB side channels?

Chapter 3

Processor Architecture and Machine Code

“The CPU is not a magic black box. It is a deterministic state machine that executes a sequence of bytes.”

Learning Goals

- Describe the x86-64 register file and its conventions.
- Read and write basic assembly instructions: `mov`, arithmetic, control flow, stack operations.
- Explain the x86-64 calling convention (System V ABI) and stack frame layout.
- Understand the relationship between C code and compiled assembly.
- Relate architectural features to exploitation primitives.

3.1 The x86-64 Register File

A processor operates on data stored in *registers*, a small, extremely fast memory bank inside the CPU. x86-64 provides 16 general-purpose registers, each 64 bits wide:

Register	Conventional name	Purpose
<code>rax</code>	Accumulator	Return value, intermediate results
<code>rbx</code>	Base	Callee-saved
<code>rcx</code>	Counter	Loop count, shift count
<code>rdx</code>	Data	Multiplication/division, I/O
<code>rsi</code>	Source index	String operations, argument #2
<code>rdi</code>	Destination index	String operations, argument #1
<code>rsp</code>	Stack pointer	Top of the stack (callee-saved)
<code>rbp</code>	Base pointer	Start of stack frame (callee-saved)
<code>r8-r15</code>	Extended registers	General purpose, argument #5-#6

Additionally, x86-64 provides:

- **Instruction pointer (rip):** address of the next instruction to execute.

- **Flags register (rflags)**: condition codes (zero, carry, overflow, sign) set by arithmetic instructions.
- **XMM registers (xmm0–xmm15)**: 128-bit registers for floating-point and SIMD operations.
- **Segment registers (cs, ds, es, fs, gs)**: used for thread-local storage and privilege level tracking.

3.1.1 Sub-Register Access

x86-64 maintains backward compatibility with earlier x86 generations. The lower portions of each register are independently addressable:

Listing 3.1: rax and its sub-registers

```
|63-----32|31-----16|15-8|7-0|
| rax (64-bit) |
|  eax (32-bit) |
|   ax (16-bit)|
|  ah | al  |
```

Writing to a 32-bit sub-register (**eax**) zero-extends to the full 64-bit register. Writing to a 16-bit or 8-bit sub-register (**ax**, **al**) preserves the upper bits.

3.2 Basic Instructions

3.2.1 Data Movement

Listing 3.2: Data movement instructions (AT&T syntax)

```
mov %rax, %rbx # rbx = rax
mov $42, %rax # rax = 42 (immediate)
mov (%rax), %rbx # rbx = *rax (load from address in rax)
mov %rax, (%rbx) # *rbx = rax (store to address in rbx)
lea (%rax,%rbx), %rcx # rcx = rax + rbx (address arithmetic)
```

The **mov** instruction copies data between registers, memory, and immediates. Crucially, x86-64 does not allow memory-to-memory moves—at most one operand can be a memory reference.

Definition 3.1 (Memory Operand). A **memory operand** is specified as **disp(base, index, scale)**, which computes $\text{address} = \text{base} + \text{index} \times \text{scale} + \text{disp}$. For example,

Chapter 4

Memory Unsafety

“A buffer overflow is not a bug. A buffer overflow is a violation of the type system at the machine level.”

Learning Goals

- Define memory unsafety and its root cause in C and C++.
- Classify the major families of memory-unsafety vulnerabilities.
- Understand stack-based buffer overflows, heap overflows, use-after-free, and format string bugs.
- Trace how each vulnerability type leads to a concrete exploitation primitive.
- Recognise patterns that indicate unsafe code during code review.

4.1 What Is Memory Unsafety?

A programming language is *memory-safe* if every memory access is guaranteed to be within the bounds of the intended object and occurs while that object is still alive. Languages like JavaScript, Python, Java, and Rust (in safe mode) enforce this guarantee—either through runtime checks, garbage collection, or a borrow checker.

C and C++ are *memory-unsafe*. They provide no automatic bounds checking, no lifetime tracking, and no protection against dangling pointers. Once a pointer is computed, the processor will dereference it regardless of whether the target is valid.

Definition 4.1 (Memory Safety Violation). A **memory safety violation** occurs when a program accesses memory outside the bounds of the intended object, or accesses memory through a pointer to an object that has already been deallocated. The C standard classifies these as *undefined behaviour*—the compiler may generate any code.

Memory-safety bugs are not merely quality issues. They are the dominant source of security vulnerabilities in systems software. Microsoft estimates that 70% of all CVEs in their products are memory-safety issues. Google finds similar numbers in Android and Chrome.

4.2 Classification of Memory Unsafety

We can classify memory unsafety along two axes: *spatial* (accessing memory beyond the object’s bounds) and *temporal* (accessing memory after the object’s lifetime):

	Spatial (out-of-bounds)	Temporal (use-after-free)
Stack	Stack buffer overflow	Return of pointer to local variable
Heap	Heap buffer overflow	Use-after-free, double-free
Global	Global buffer overflow	N/A (global lifetime)

4.3 Stack Buffer Overflows

The stack buffer overflow is the classic memory-safety vulnerability, documented by Aleph One in 1996 [1].

Listing 4.1: Vulnerable stack buffer

```
void vulnerable(char *input) {
    char buf[64];
    strcpy(buf, input); // No bounds check!
}
```

The function allocates 64 bytes on the stack. If `input` is longer than 63 characters (plus null terminator), `strcpy` writes past the end of `buf`, overwriting the saved `rbp` and the return address.

Principle 4.1 (The Exploitation Primitive). Overwriting the return address gives the attacker control of the instruction pointer (`rip`) when the function returns. The attacker redirects execution to their shellcode or a ROP chain.

The layout just before `strcpy` writes:

Listing 4.2: Stack layout during overflow

```
High addresses
+-----+
| Return address | <- overwritten by attacker
+-----+
| Saved rbp | <- overwritten
+-----+
| buf[63..56] | <- top of buffer
| buf[55..48] |
| ... |
| buf[7..0] | <- bottom of buffer (buf[0])
+-----+ <- rsp
Low addresses
```

4.3.1 Functions That Enable Overflows

Certain C library functions are notorious for lacking bounds checking:

Function	Problem	Safe alternative
<code>strcpy(dst, src)</code>	No length argument	<code>strncpy</code> , <code>strlcpy</code> , <code>memcpy</code>
<code>sprintf(buf, fmt, ...)</code>	No buffer size	<code>snprintf</code>
<code>gets(buf)</code>	No limit	<code>fgets</code>

<code>scanf("%s",</code>	No field width	<code>scanf("%Ns", buf)</code>
<code>buf)</code>		
<code>memcpy(dst,</code>	Relies on caller for <code>n</code>	Manual bounds check
<code>src, n)</code>		

4.4 Heap Overflows

Heap overflows are analogous to stack overflows but occur in dynamically allocated memory.

Listing 4.3: Vulnerable heap buffer

```
void heap_vulnerable(char *input) {
    char *buf = malloc(64);
    strcpy(buf, input); // If input > 63 bytes, overflow
}
```

Heap exploitation is more complex because the target is not a fixed return address but the heap metadata or adjacent heap objects. Modern heap allocators use complex data structures (bins, fastbins, tcache) that can be corrupted to achieve arbitrary write.

Definition 4.2 (Heap Metadata Corruption). Heap allocators store metadata (chunk size, flags, free-list pointers) in-band with user data. Overwriting this metadata can cause the allocator to write attacker-controlled values to attacker-controlled addresses—a *write-what-where* primitive.

A typical heap chunk layout (in `ptmalloc`):

Listing 4.4: Heap chunk structure (simplified)

```
Low address
+-----+
| prev_size (8 bytes) | <- size of previous chunk
+-----+
| size (8 bytes) | <- current chunk size + flags
+-----+
| user data ... | <- what malloc() returns
| ... |
+-----+
High address
```

Overflowing from chunk A into chunk B can corrupt B's size field or, if B is freed, its free-list pointers. When B is later allocated, the allocator dereferences the corrupted pointers, giving the attacker a controlled write.

4.5 Use-After-Free

A **use-after-free** (UAF) bug occurs when memory is accessed through a pointer after it has been freed.

Listing 4.5: Use-after-free example

```
struct object *obj = malloc(sizeof(*obj));
// ... use obj ...
free(obj);
// ... obj is now a dangling pointer ...
obj->field = 42; // Use-after-free! Writing to freed memory
```

After **free**, the memory at `obj` may be reused by a subsequent allocation. If an attacker can control the data in that new allocation (e.g., by allocating a buffer of the same size with attacker-controlled content), they can manipulate the original object's fields.

Definition 4.3 (Dangling Pointer). A **dangling pointer** is a pointer that references memory that has been freed. Dereferencing a dangling pointer is undefined behaviour that typically results in a use-after-free vulnerability.

UAF exploitation often involves:

1. Allocating an object of type **A**.
2. Freeing it while retaining a pointer.
3. Allocating a controlled buffer of the same size (which gets the freed **A**'s memory).
4. Accessing the original pointer, which now reads/writes attacker-controlled data.

4.5.1 Double Free

A related temporal bug is the **double free**: freeing the same pointer twice. This corrupts the allocator's free-list data structures.

Listing 4.6: Double free

```
void *p = malloc(64);
free(p);
free(p); // Double free!
```

After the first **free**, the allocator adds `p` to the free list. After the second **free**, `p` is added again. Subsequent allocations may return the same memory twice, bypassing the allocator's integrity guarantees.

4.6 Format String Vulnerabilities

Format string bugs are an entirely different class of memory unsafety.

Listing 4.7: Format string vulnerability

```
void vulnerable(char *user_input) {
    printf(user_input); // Attacker controls the format string!
}
```

The correct call is `printf("%s", user_input)`. When the attacker supplies a format string like `"%x %x %x %x %x"`, `printf` reads arguments from the stack that were never passed, leaking memory contents.

More dangerously, the `%n` specifier writes the number of bytes printed so far to a pointer argument:

Listing 4.8: Using `%n` to write memory

```
// If the attacker can supply "%n" to printf,
// printf will write 4 bytes to the address on the stack
// where it expects the next argument.
printf("%x%x%x%n"); // writes to stack, corrupting memory
```

This gives the attacker an arbitrary-write primitive by carefully controlling the number of printed bytes and the target address (planted on the stack by preceding format arguments).

Specifier	Effect	Security relevance
-----------	--------	--------------------

%x	Read unsigned hex from stack	Stack memory leak
%s	Read string from stack pointer	Read memory at arbitrary address
%p	Read pointer from stack	Leak addresses (bypass ASLR)
%n	Write count to pointer on stack	Arbitrary memory write
%hn	Write 2 bytes (short)	Fine-grained memory write
%hhn	Write 1 byte (char)	Byte-level memory write

4.7 Integer Overflows Leading to Memory Unsafety

Integer overflows are safety bugs that often mask memory-safety vulnerabilities:

Listing 4.9: Integer overflow enabling overflow

```
void vulnerable(size_t size) {
    char *buf = malloc(size + 1); // If size = SIZE_MAX, size+1 = 0!
    read(fd, buf, size); // Overflow into adjacent heap memory
}
```

If `size` is `SIZE_MAX`, then `size + 1` wraps to 0. `malloc(0)` returns a valid (but tiny) buffer. The subsequent `read` writes `SIZE_MAX` bytes into that small buffer.

4.8 The Eternal War

Szekeres et al. [5] argue that memory corruption is an *undecidable* problem in general: no static analysis can prove the absence of all memory-safety bugs in arbitrary C/C++ programs. The best we can do is:

Approach	Mechanism	Limitation
Static analysis	Symbolic execution, abstract interpretation	False positives, incomplete coverage
Dynamic analysis	AddressSanitizer, valgrind	Runtime overhead, no coverage of unexecuted paths
Memory-safe languages	Type systems, borrow checker	Runtime overhead (GC), ecosystem friction
Hardware protection	NX, ASLR, MPK	Doesn't prevent the bug, only limits exploitation

4.9 Exercises

Exercise 4.1. *Identify the memory-safety bug in this code. What is the vulnerability class? How would you fix it?*

```
void copy_name(char *name) {
    char buf[16];
    strcpy(buf, name);
    printf("Name: %s\n", buf);
}
```

Exercise 4.2. *What is the output of this program? What security implication does it have?*

```
int main() {  
    char *s = malloc(8);  
    free(s);  
    printf("%p\n", s);  
}
```

Exercise 4.3. *Given `printf(user_input)`, how would an attacker leak a secret value at address `0x7fffffff000` using format string specifiers?*

Exercise 4.4. *Research CVE-2021-26708 (a Linux kernel use-after-free). Describe the bug, the exploit technique, and the patch. What allocation type was reused?*

Part II

Exploitation and Defence

Chapter 5

Exploitation Techniques

“An exploit is not magic. It is a precise sequence of machine operations that follows the same rules as any other program.”

Learning Goals

- Write and understand x86-64 shellcode for common primitives.
- Construct a stack-based buffer overflow exploit with controlled code execution.
- Understand return-oriented programming (ROP) and its variants.
- Explain how format string exploits achieve arbitrary read and write.
- Trace the evolution of exploitation techniques in response to mitigations.

5.1 From Bug to Exploit

A memory-corruption bug gives the attacker a *primitive*: the ability to read or write memory beyond intended bounds. Exploitation is the process of elevating that primitive into full control over the victim process.

Definition 5.1 (Exploitation Primitive). An **exploitation primitive** is a low-level capability obtained through a vulnerability. Common primitives include:

- **Arbitrary read:** read any address in the process’s address space.
- **Arbitrary write:** write any value to any address.
- **Control flow hijack:** redirect the instruction pointer to an attacker-controlled address.

5.2 Shellcode

Shellcode is position-independent machine code that performs a desired action. The name comes from the classic goal: spawn a shell (`/bin/sh`). Modern shellcode can do anything—execute a command, establish a reverse connection, or patch memory.

Definition 5.2 (Shellcode). **Shellcode** is a sequence of machine-code bytes designed to be injected into a process and executed. It must be *position-independent* (no absolute addresses) and typically avoids null bytes (which would terminate string-based injection vectors).

5.2.1 A Minimal Shellcode: `execve("/bin/sh")`

The classic shellcode executes `execve("/bin/sh", NULL, NULL)`:

Listing 5.1: x86-64 `execve` shellcode

```
; execve("/bin/sh", NULL, NULL)
xor %rdx, %rdx # rdx = 0 (envp = NULL)
xor %rsi, %rsi # rsi = 0 (argv = NULL)
mov $0x68732f6e69622f, %rax # rax = "/bin//sh" (little-endian)
push %rax
mov %rsp, %rdi # rdi = pointer to "/bin//sh"
xor %rax, %rax
mov $59, %al # rax = 59 (execve syscall number)
syscall
```

The bytes (27 bytes, null-free):

```
48 31 d2 48 31 f6 48 b8 2f 62 69 6e 2f 2f 73 68
50 48 89 e7 48 31 c0 b0 3b 0f 05
```

Note the use of `/bin//sh` (with double slash) to produce an 8-byte value that fits in a register. The double slash is harmless: the kernel normalises it to `/bin/sh`.

5.2.2 Null-Free Constraint

Many injection vectors (e.g., `strcpy`, `sprintf`) stop copying at the first null byte. Shellcode must not contain `0x00`. Common techniques:

- Use `xor reg, reg` instead of `mov reg, 0`.
- Use `mov $0xFF, %al; inc %al` instead of `mov $0, %al`.
- Use `shl` to construct constants without null bytes.

Example 5.1 (Null-Free xor Trick). To zero a register: `xor %rax, %rax` (2 bytes: `48 31 c0`). To set `rax = 59`: `xor %rax, %rax; mov $59, %al` (5 bytes: `48 31 c0 b0 3b`).

5.3 Classic Stack Smashing

With no mitigations (no NX, no ASLR, no stack canary), exploiting a stack buffer overflow is straightforward:

Listing 5.2: Classic exploitation layout

```
Buffer address: 0x7fffffff000
Return address: 0x7fffffff040

[AAAA...AAAA] [BBBBBBBB] [CCCCCCCC]
|-- shellcode |--| rbp |--| return address |--|

return address = buffer_address (= 0x7fffffff000)
```

The attacker:

1. Fills the buffer with shellcode (padded to the exact buffer size).
2. Overwrites saved `rbp` (can be arbitrary, 8 bytes).
3. Overwrites the return address with the buffer's address.

When `ret` executes, `rip` jumps to the shellcode.

5.3.1 NOP Sleds

In practice, the attacker may not know the exact buffer address. A *NOP sled* mitigates this: a sequence of `nop` (0x90) instructions before the shellcode:

Listing 5.3: NOP sled

```
[90 90 90 ... 90] [shellcode] [overwritten rbp] [target address]
|-- NOP sled (~half the buffer) --|
```

If `rip` lands anywhere in the NOP sled, execution slides into the shellcode. The sled increases the probability of successful exploitation by a factor equal to the sled length divided by the page size.

5.4 Return-to-libc (ret2libc)

When NX (2) prevents execution of shellcode on the stack, attackers turn to *code reuse*. Return-to-libc redirects execution to existing library functions.

Listing 5.4: ret2libc exploit layout

```
[padding to fill buffer] [saved rbp] [return address] [argument 1] [argument 2]

return address = address of system() in libc
argument 1 = address of "/bin/sh" string in libc
```

The stack is crafted so that when `vulnerable` returns, `rip` lands at `system()`, and the stack contains the argument pointer. This calls `system("/bin/sh")` without injecting any code.

Definition 5.3 (Return-to-libc (ret2libc)). A code-reuse technique that hijacks the return address to call an existing library function (commonly `system`) with attacker-controlled arguments on the stack.

5.5 Return-Oriented Programming (ROP)

ret2libc calls a single function. Return-Oriented Programming [4] chains multiple short instruction sequences (*gadgets*), each ending with `ret`, to perform arbitrary computation.

Definition 5.4 (ROP Gadget). A **ROP gadget** is a sequence of a few instructions ending with `ret` (0xC3). Gadgets are found within existing executable code (libc, the binary itself). The attacker chains gadgets by placing their addresses sequentially on the stack.

Each `ret` pops the next gadget address from the stack and jumps to it. The ROP chain is essentially a program written in stack entries:

Listing 5.5: ROP chain on the stack

```
High addresses
+-----+
| gadget_3_address | <- ret from gadget_2 jumps here
+-----+
| argument for gadget_2 |
+-----+
| gadget_2_address | <- ret from gadget_1 jumps here
+-----+
| argument for gadget_1 |
+-----+
| gadget_1_address | <- overwritten return address
```

```
+-----+
Low addresses
```

5.5.1 Finding Gadgets

Gadgets are found by scanning executable memory for `ret` (0xC3) bytes and looking backward for useful instruction sequences. Tools like `ROPgadget`, `ropper`, and `radare2` automate this:

```
$ r2 /bin/ls
[0x100001180]> /R ret
...
0x100003510: 58 59 c3 (pop rax; pop rcx; ret)
0x100003520: 5b c3 (pop rbx; ret)
...
```

A useful ROP chain typically requires gadgets for:

- Loading values into registers (`pop reg; ret`).
- Writing to memory (`mov [reg], reg; ...; ret`).
- Performing syscalls (`syscall; ret`).
- Adjusting the stack pointer (`add rsp, N; ret`).

5.5.2 Example: ROP `execve`

A minimal ROP chain to call `execve("/bin/sh", NULL, NULL)` on x86-64:

Listing 5.6: Conceptual ROP chain

```
Stack layout (growing upward):
Address of: pop_rdi; ret # gadget to pop rdi
Address of: "/bin/sh" string # value loaded into rdi
Address of: pop_rsi; ret # gadget to pop rsi
0 # rsi = NULL
Address of: pop_rdx; ret # gadget to pop rdx (if available)
0 # rdx = NULL
Address of: pop_rax; ret # gadget to pop rax
59 # rax = execve syscall number
Address of: syscall; ret # invoke syscall
```

5.6 Format String Exploitation

Format string bugs provide both an information leak and a write primitive.

5.6.1 Memory Leak

The attacker supplies `%p.%p.%p.%p` to `printf`. Each `%p` reads and prints a 64-bit value from the stack:

Listing 5.7: Leaking stack values

```
Input: %p.%p.%p.%p.%p.%p
Output: 0x7fffffff00000000.0x7f00000000fc.(nil).0x7ffff7a...
```

By positioning the format string buffer on the stack, the attacker can read specific stack positions. The `%N$p` syntax reads the *N*th argument directly:

Listing 5.8: Direct parameter access

Input: %6\$p # read the 6th stack argument
 Output: 0x7ffff7dd0b80 # likely a libc address (ASLR leak)

Leaking a libc address defeats ASLR (6): the attacker computes the base address and derives addresses of `system`, gadgets, and other targets.

5.6.2 Arbitrary Write with %n

The `%n` specifier writes the number of bytes output so far to a pointer on the stack. The attacker places the target address on the stack (within the format buffer) and uses `%N$n` to write to it:

Listing 5.9: Writing to an arbitrary address

```
// Target address (8 bytes in little-endian) placed at the
// start of the format buffer, followed by format specifiers.
// \x00\xe0\xff\xff\xff\xff\x00\x00 = 0x7fffffff000
// Then: %125c%6$hhn writes 125 to byte at 0x7fffffff000

// %125c prints 125 characters (sets count to 125)
// %6$hhn writes the count (125 = 0x7d) as a single byte
// to the address at the 6th argument position
```

Multiple `%hhn` writes are combined to write a full 64-bit pointer one byte at a time.

5.7 Heap Exploitation

Heap exploitation is more complex than stack exploitation and depends on the specific allocator. Common techniques include:

Technique	Mechanism	Target
Tcache poisoning	Overwrite tcache next pointer	Arbitrary allocation
Fastbin attack	Corrupt fastbin free list	Arbitrary write
House of Force	Overflow into top chunk size	Arbitrary allocation
Use-after-free	Reallocate freed object	Type confusion
Unsorted bin attack	Write to unsorted bin bk pointer	Arbitrary write via un-link

Principle 5.1 (Heap Exploitation Is Allocator-Specific). Heap exploitation techniques are tightly coupled to a particular allocator implementation and version. The transition from `glibc ptmalloc` to `jemalloc` or `mimalloc` completely changes the attack surface.

5.8 The Exploitation Pipeline

Modern exploitation against hardened systems follows a general pipeline:

1. **Information leak:** leak a code or libc pointer to defeat ASLR.
2. **Address computation:** compute base addresses and gadget locations.
3. **Memory corruption:** trigger the vulnerability to gain an arbitrary write.
4. **Code execution:** overwrite a function pointer, return address, or vtable with a ROP chain.
5. **Payload delivery:** execute the final shellcode (via `mprotect` + NX bypass or pure ROP).

5.9 Exercises

Exercise 5.1. *Modify the `execve` shellcode to execute `/bin/cat /etc/passwd` instead of `/bin/sh`. What changes are needed?*

Exercise 5.2. *Given a buffer of 128 bytes on the stack with NX enabled and ASLR disabled, construct a `ret2libc` payload that calls `system("/bin/sh")`. Assume `system` is at `0x7ffff7a3d580` and the string `"/bin/sh"` is at `0x7ffff7b99e17`.*

Exercise 5.3. *In a format string exploit, what does `%100c%12$hn` do? If the target address (placed at position 12 on the stack) is `0x7fffffffe000`, what value is written?*

Exercise 5.4. *Research the Blinding ROP technique [6]. How does it bypass ASLR without an information leak?*

Chapter 6

Protections and Mitigations

“Every mitigation raises the cost of exploitation. None of them makes exploitation impossible.”

Learning Goals

- Describe the major defensive mechanisms in modern operating systems and compilers.
- Explain how each mitigation disrupts specific exploitation techniques.
- Understand the interaction between mitigations and the exploitation pipeline.
- Compare hardware, compiler, and OS-level protection strategies.
- Evaluate the effectiveness and limitations of current mitigations.

6.1 The Mitigation Stack

Modern systems deploy defences at multiple layers:

Layer	Mitigation	What it prevents
Hardware	NX bit	Direct shellcode execution
Hardware	SMEP/SMAP	Kernel-mode code reuse from user pages
OS	ASLR	Predictable address computation
OS	KASLR	Kernel address prediction
Compiler	Stack canary	Stack buffer overflow detection
Compiler	CFI	Arbitrary control flow hijacking
Compiler	SafeStack	Separate stack for sensitive data
Runtime	AddressSanitizer	Heap/stack/bounds violation detection
Runtime	seccomp	System call filtering

6.2 Non-Executable Memory (NX/DEP)

The NX (No-Execute) bit, called Data Execution Prevention (DEP) on Windows, marks memory pages as non-executable. This prevents the classic exploitation technique of writing shellcode to the stack or heap and executing it.

Definition 6.1 (NX/DEP). **NX** is a hardware feature that allows each page table entry to specify whether the page is executable. The processor checks this bit on every instruction fetch; if **NX=1** and the processor is not in supervisor mode, a page fault is raised.

NX is nearly universal on modern systems. It is not a complete defence—attackers use code-reuse techniques (ret2libc, ROP) instead—but it eliminates the simplest exploitation path.

Remark 6.1. Some JIT compilers (e.g., JavaScript JITs in browsers) need executable memory at runtime. They use `mprotect` or `VirtualProtect` to mark pages as executable, creating a window of opportunity if an attacker can corrupt JIT code.

6.3 Address Space Layout Randomisation (ASLR)

ASLR randomises the base address of each memory region (stack, heap, libraries, executable) when a process starts. This prevents the attacker from knowing the absolute address of gadgets, shellcode, or target buffers.

Definition 6.2 (ASLR). **Address Space Layout Randomisation** is a technique that randomises the virtual address at which each memory region is mapped. The randomness is typically on the order of bits (e.g., 28 bits for the `mmap` base on x86-64 Linux), making it infeasible to guess the correct address.

6.3.1 ASLR Effectiveness

ASLR is effective against attacks that require absolute addresses. The probability of a successful guess is:

$$P_{\text{success}} = \frac{1}{2^N}$$

where N is the number of entropy bits. On x86-64 Linux:

- Stack: 22 bits of entropy (aligned to 16 bytes).
- `mmap` (shared libraries): 28 bits.
- Heap: 13 bits.
- Executable: 0 bits by default (but PIE executables add 28 bits).

A brute-force attempt that successfully guesses an address with 2^{28} possibilities on the first try has probability 3.7×10^{-9} .

6.3.2 Bypassing ASLR

Attackers bypass ASLR through *information leaks*: any vulnerability that reveals a pointer or code address. Common leaks:

- Format string bugs (`%p`) leak stack and code addresses.
- Heap overflows that corrupt metadata and leak pointers through allocator operations.
- Side-channel attacks (Spectre) that leak address information through cache timing.
- Uninitialised memory that contains stale pointer values.

Principle 6.1 (ASLR + Info Leak = No ASLR). ASLR is only as strong as the information hiding that supports it. A single leaked pointer renders ASLR ineffective for the entire address-space region containing that pointer.

6.3.3 Position-Independent Executables (PIE)

A Position-Independent Executable (PIE) compiles the main binary as position-independent code, enabling ASLR for the executable itself. Without PIE, the executable's code is at a fixed address, providing stable targets for ROP gadgets.

Listing 6.1: Checking if a binary is PIE

```
$ readelf -h /bin/ls | grep Type
Type: DYN (Shared object file) # PIE
$ readelf -h /bin/cat | grep Type
Type: EXEC (Executable file) # Not PIE (legacy)
```

Most modern distributions compile all executables as PIE.

6.4 Stack Canaries

A stack canary is a random value placed between local variables and the saved return address on the stack. Before the function returns, the canary is checked. If it has changed, a buffer overflow has occurred and the program aborts.

Definition 6.3 (Stack Canary). A **stack canary** is a secret value written into the stack frame during the function prologue and verified before the epilogue. Corruption of the canary indicates a stack buffer overflow.

Listing 6.2: Canary-protected stack frame

```
void vulnerable(char *input) {
    char buf[64];
    // Compiler inserts:
    // canary = __readgsqword(0x28); // read from TLS
    // ... store canary on stack ...
    strcpy(buf, input);
    // ... check canary before ret ...
    // if (canary != __stack_chk_guard) __stack_chk_fail();
}
```

The canary is:

- Random: different per process, different per thread.
- Secret: stored in thread-local storage (TLS).
- Terminated with a null byte: prevents string-based leaks.

The null-byte terminator means that a canary byte cannot be 0x00. If an overflow is done byte-by-byte (e.g., one byte past a buffer), the attacker would overwrite the null byte, which is detected. More importantly, the null byte prevents `strcpy` and similar functions from reading past the canary.

6.4.1 Bypassing Stack Canaries

Stack canaries are bypassed by:

- **Information leaks:** if the canary value is leaked (e.g., through a format string bug), the attacker can include the correct canary in the overflow payload.
- **Overwrite-before-check:** some code paths allow overwriting a function pointer *before* the return (e.g., overflow within the same frame that corrupts a local function pointer).
- **Non-stack overflows:** canaries only protect stack frames. Heap overflows, use-after-free, and format string bugs are unaffected.

6.5 Control-Flow Integrity (CFI)

Control-Flow Integrity [3] restricts the set of targets to which indirect branches (function pointers, vtables, return addresses) can point.

Definition 6.4 (CFI). **Control-Flow Integrity** ensures that every indirect control transfer (call, jump, return) goes to a target that was valid at compile time. A *forward-edge* CFI constrains calls and jumps; a *backward-edge* CFI constrains returns.

6.5.1 Forward-Edge CFI

Forward-edge CFI checks that indirect calls and jumps target a valid set of addresses. The compiler determines, for each call site, the set of functions that could legally be reached (based on type information and control-flow analysis). At runtime, before the indirect branch, the target is validated against this set.

Implementation approaches:

- **Clang CFI:** uses type-based analysis to restrict indirect call targets at link time. Overhead is typically 1–5%.
- **Microsoft Control Flow Guard (CFG):** checks indirect call targets against a bitmap of valid function addresses. Overhead is minimal.
- **Intel CET (Control-flow Enforcement Technology):** hardware-assisted CFI using a shadow stack (backward-edge) and indirect branch tracking (IBT, forward-edge).

6.5.2 Backward-Edge CFI (Shadow Stacks)

A shadow stack is a separate, protected memory region that stores a copy of each return address. On function return, the processor compares the return address on the data stack with the shadow stack:

Listing 6.3: Shadow stack concept

```
; Function prologue
push return_addr ; on normal stack
push return_addr ; on shadow stack (protected)

; Function epilogue
pop shadow_addr ; from shadow stack
pop return_addr ; from normal stack
if (shadow_addr != return_addr) fault();
```

Shadow stacks prevent ROP (which rewrites return addresses on the data stack) but require hardware support (Intel CET Shadow Stack) or significant software overhead (software shadow stacks like `retpoline`).

6.6 Compiler Hardening

Modern compilers offer numerous security hardening options:

Flag	Effect	Overhead
<code>-fstack-protector</code>	Canary for functions with local buffers	Minimal
<code>-fstack-protector-strong</code>	Canary for any function with local array	Slightly more

<code>-fstack-clash-protection</code>	Prevent stack clash	Small probing
<code>-D_FORTIFY_SOURCE=2</code>	Bounds check for <code>__builtin_memcpy</code> etc.	Near zero
<code>-fPIE -pie</code>	Position-independent executable	Minimal
<code>-Wl,-z,relro</code>	Read-only relocations after loading	None
<code>-Wl,-z,now</code>	Full RELRO (resolve at load time)	Startup cost
<code>-fsanitize=address</code>	Full runtime bounds + UAF detection	2–3x slowdown

6.7 Runtime Defences

6.7.1 seccomp

seccomp (secure computing mode) is a Linux kernel feature that restricts which system calls a process can make. A process can install a seccomp filter (written as a BPF program) that is evaluated on every `syscall`:

Listing 6.4: Minimal seccomp filter

```
#include <seccomp.h>

scmp_filter_ctx ctx = seccomp_init(SCMP_ACT_KILL);
seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(read), 0);
seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(write), 0);
seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(exit), 0);
seccomp_load(ctx);
```

Even if an attacker achieves arbitrary code execution, seccomp restricts what they can do. For example, blocking `execve` prevents shell spawning; blocking `mprotect` prevents making NX pages executable.

6.7.2 Sanitizers

Sanitizers are runtime instrumentation tools that detect memory safety violations:

- **AddressSanitizer (ASan)**: detects heap and stack buffer overflows, use-after-free, and double free. Uses shadow memory to track valid memory regions.
- **MemorySanitizer (MSan)**: detects reads of uninitialised memory.
- **UndefinedBehaviorSanitizer (UBSan)**: detects various forms of undefined behaviour (integer overflow, misaligned access, etc.).
- **LeakSanitizer (LSan)**: detects memory leaks.

Sanitizers impose runtime overhead (2–5x for ASan) and are intended for testing, not production deployment.

6.8 The Arms Race

Mitigations and bypass techniques co-evolve:

Mitigation	Bypass	Countermeasure
NX/DEP	ROP	CFI, shadow stack

ASLR	Information leak	Efficient randomization, zero-leak primitives
Stack canary	Thread-local storage leak	SafeStack, canary rotation
PIE	Partial overwrite	ASLR with high entropy
CFI (coarse)	Out-of-bounds target call	Fine-grained CFI, CET IBT
seccomp	Syscall number mutation	BPF verification of syscall args
KASLR	Prefetch side channel	KPTI, FG-KASLR

Principle 6.2 (Defence in Depth). No single mitigation is sufficient. The goal of defence in depth is to require multiple independent bypasses for a successful exploit. A ROP chain bypasses NX but requires an ASLR leak; a CFI bypass requires a different vulnerability class.

6.9 Exercises

Exercise 6.1. *Given a binary compiled with `-fstack-protector` but no PIE and no ASLR, design an exploitation strategy. Which bypass techniques are unnecessary? Which remain?*

Exercise 6.2. *Ubuntu 22.04 ships with `-fstack-protector-strong` and PIE by default. A local service has a heap overflow in a function that does not use stack buffers. Which mitigations stop the attacker? Which are irrelevant?*

Exercise 6.3. *Read about Intel CET. What two components does it provide? How does the shadow stack prevent ROP, and why does it require a new processor instruction?*

Exercise 6.4. *Design a seccomp filter for a web server. Which syscalls must be allowed? Which can be safely blocked? Would your filter prevent an attacker who achieves RCE from spawning a reverse shell?*

Chapter 7

Kernel Security

“The kernel is not a trust boundary. It is the trust anchor.”

Learning Goals

- Describe the privilege model of modern operating systems (rings, modes).
- Understand the kernel attack surface: system calls, device drivers, `ioctl`.
- Classify kernel vulnerability types and their exploitation primitives.
- Explain kernel defences: KASLR, SMEP, SMAP, KPTI, CFI.
- Trace the exploitation pipeline for a typical kernel vulnerability.

7.1 Privilege Rings

Processors enforce privilege through *protection rings*. x86-64 provides four rings (0–3), though most operating systems use only two:

- **Ring 0 (Kernel mode):** full access to all memory, I/O ports, and privileged instructions. The operating system kernel and device drivers run here.
- **Ring 3 (User mode):** restricted access. User-mode processes cannot execute privileged instructions (like `lgdt`, `lidt`), cannot access kernel memory, and cannot mask interrupts.

Definition 7.1 (Privilege Escalation). **Privilege escalation** is the act of gaining higher privileges than those initially granted. In the context of kernel security, it typically means going from Ring 3 (user) to Ring 0 (kernel) code execution.

The transition from user to kernel mode occurs only through controlled entry points: system calls, interrupts, and exceptions. The processor switches the privilege level and jumps to a handler whose address is stored in the Interrupt Descriptor Table (IDT) or via the `syscall` instruction (which reads from Model-Specific Registers).

7.2 The Kernel Attack Surface

A kernel vulnerability is a bug in kernel code that can be triggered from user space. The attack surface includes:

7.2.1 System Calls

Every system call is an entry point from user to kernel space. Linux has over 400 syscalls; each is a potential attack vector. A bug in any syscall implementation (e.g., a missing bounds check in `ioctl`) is a kernel vulnerability.

7.2.2 Device Drivers

Device drivers account for the majority of kernel code and a disproportionate share of vulnerabilities. Drivers are often written by third-party vendors with less rigorous review, and they operate with full kernel privileges.

Listing 7.1: Simplified vulnerable driver `ioctl`

```
static long driver_ioctl(struct file *file, unsigned int cmd,
                        unsigned long arg) {
    char buf[64];
    switch (cmd) {
        case CMD_WRITE:
            if (copy_from_user(buf, (void __user *)arg, sizeof(buf)))
                return -EFAULT;
            // ... use buf ...
            break;
        case CMD_WRITE_LARGE:
            // BUG: no size check!
            copy_from_user(buf, (void __user *)arg, arg_size);
            break;
    }
}
```

The `CMD_WRITE_LARGE` path overflows the kernel stack buffer—a classic kernel stack buffer overflow.

7.2.3 `procfs` and `sysfs`

Pseudo-file systems `/proc` and `/sys` expose kernel-internal data to user space. Reading or writing specific files invokes kernel callbacks. Bugs in these callbacks (race conditions, missing validation) are common.

7.2.4 `ioctl`

The `ioctl` system call on device files is notoriously complex. Each driver defines its own commands, argument structures, and validation logic—a rich source of vulnerabilities.

7.3 Kernel Vulnerability Classes

7.3.1 Kernel Stack Overflow

Analogous to user-space stack overflows but more severe: overflowing the kernel stack corrupts kernel control data.

Listing 7.2: Kernel stack overflow

```
asmlinkage long sys_buggy_ioctl(unsigned int fd, unsigned int cmd,
                               unsigned long arg) {
    char buf[64];
    // arg_size can be > 64!
    if (copy_from_user(buf, (void __user *)arg, arg_size))
```



```

    return -EFAULT;
}

```

Kernel stack is typically small (8–16 KB) and cannot grow. Stack overflows in the kernel are especially dangerous.

7.3.2 Kernel Heap Overflow

The kernel has its own allocators (slab, slub, slob). Heap overflows corrupt kernel objects:

Listing 7.3: Kernel heap overflow

```

struct cred *cred = prepare_creds();
// cred is a kernel object allocated from slab
// Overflowing an adjacent slab object can overwrite cred->uid

```

7.3.3 Use-After-Free in Kernel

Kernel UAF occurs when a kernel object is freed but a reference remains:

Listing 7.4: Kernel use-after-free

```

struct file *file = fget(fd);
// ... file might be closed and freed by another thread ...
fput(file);
// UAF: file->f_op->write(file, ...)

```

The classic kernel UAF pattern: race between two threads where one frees an object and the other accesses it.

7.3.4 Double Fetch

A double fetch occurs when kernel code reads user-space data twice, and the data changes between reads (a TOCTOU race):

Listing 7.5: Double fetch vulnerability

```

long fixup_ioctl(void __user *arg) {
    struct user_data data;
    copy_from_user(&data, arg, sizeof(data)); // First fetch
    int size = data.size;
    // ... validation passes ...
    copy_from_user(kbuf, arg->buf, data.size); // Second fetch!
    // data.size may have changed!
}

```

A malicious thread modifies `data.size` between the two `copy_from_user` calls, bypassing the validation.

7.4 Kernel Exploitation Primitives

Kernel exploits aim to elevate privileges (get root) rather than spawn a shell. Common primitives:

Primitive	Mechanism	Goal
<code>commit_creds(prepare_kernel_cred(0))</code>	Call these kernel functions	Root credentials

Modify <code>cred</code> struct	Overwrite <code>uid=0, gid=0</code> in process creds	Root
Modify <code>task_struct</code>	Overwrite <code>real_cred</code> pointer	Process creds
MMU manipulation	Modify page tables	Code execution
<code>modprobe_path</code>	Overwrite path to execute as root	Kernel-mode code exec

7.4.1 `commit_creds` Exploit

The classic kernel exploit pattern:

1. Gain arbitrary kernel memory write (via heap overflow, UAF, etc.).
2. Write the address of `commit_creds` and `prepare_kernel_cred(0)` into the kernel's function pointer table or overwrite a process's `cred` structure.
3. Trigger execution flow that calls these functions.
4. The current process now has root credentials.

7.4.2 `Modprobe_Path` Technique

A simpler technique that requires only an arbitrary write:

1. Write to `/proc/sys/kernel/modprobe_path` (or directly to kernel memory at the `modprobe_path` symbol) to point to a script you control.
2. Execute an unknown binary format, causing the kernel to invoke `modprobe_path` as root.
3. Your script runs as root.

7.5 Kernel Defences

7.5.1 KASLR

Kernel Address Space Layout Randomisation (KASLR) randomises the base address where the kernel image is loaded. On x86-64, the kernel text is typically mapped at a random offset within a large region.

7.5.2 SMEP and SMAP

Definition 7.2 (SMEP and SMAP). **SMEP** (Supervisor Mode Execution Prevention) prevents the kernel from executing code in user-space pages. **SMAP** (Supervisor Mode Access Prevention) prevents the kernel from accessing user-space data (except through explicit `copy_from/to_user`).

SMEP forces kernel exploits to use ROP within kernel code (cannot redirect to user-space shellcode). SMAP prevents direct dereferencing of user-space pointers from kernel mode.

7.5.3 KPTI

Kernel Page Table Isolation (KPTI) separates kernel and user page tables. In user mode, only a minimal set of kernel pages are mapped (just enough to handle syscalls and interrupts). This mitigates Meltdown-style side-channel attacks and reduces the kernel's attack surface.

7.5.4 Kernel CFI

Clang’s CFI can be applied to the kernel (CFI_CLANG in Linux). Forward-edge CFI for the kernel restricts indirect call targets to valid functions, preventing many control-flow hijack exploits.

7.5.5 seccomp for the Kernel

While seccomp is a user-space defence, it also protects the kernel by reducing the syscall surface that attackers can use. A restricted seccomp profile means fewer kernel code paths are reachable from a compromised process.

7.6 The Kernel Exploit Pipeline

Modern kernel exploitation follows a pattern:

1. **Gather information:** probe KASLR offset via side channels or uninitialised memory leaks.
2. **Prepare user space:** mmap user pages at predictable addresses for ROP chains or data structures.
3. **Trigger vulnerability:** invoke the buggy syscall or `ioctl` to gain a write primitive.
4. **Bypass SMAP/SMEP:** use kernel ROP (gadgets within kernel code) or overwrite function pointers.
5. **Escalate privileges:** call `commit_creds(prepare_kernel_cred(0))` via ROP or function pointer hijack.
6. **Restore execution:** return to user mode with root privileges.

Principle 7.1 (User-to-Kernel-to-User). Exploitation does not stop at kernel code execution. The exploit must return control to user space with the privilege elevation applied. This requires saving and restoring user-space register state and the stack pointer—a significant engineering challenge.

7.7 Exercises

Exercise 7.1. *Explain why SMEP does not prevent ROP within the kernel. What does SMEP prevent that ROP cannot bypass?*

Exercise 7.2. *A Linux device driver has a heap overflow in an `ioctl` handler. The kernel is compiled with `SLAB_FREELIST_RANDOM` and `CONFIG_SLUB`. What additional information does an attacker need to exploit this?*

Exercise 7.3. *Read about CVE-2022-0847 (Dirty Pipe). What was the vulnerability? What privilege did it grant? How did it bypass existing mitigations?*

Exercise 7.4. *Design a seccomp filter for a process that only needs to read files (`read`, `openat`, `mmap`, `munmap`, `write` for `stdout`). Would this prevent exploitation of a kernel UAF bug triggered from this process? Why or why not?*

Part III

Applied Low-Level Security

Chapter 8

Network Security from the Low-Level Perspective

“The network is not a source of truth.
The network is a source of bytes.”

Learning Goals

- Understand network packet structure at the byte level (Ethernet, IP, TCP/UDP headers).
- Identify low-level vulnerabilities in network protocol implementations.
- Explain packet injection, socket-level attacks, and kernel-bypass networking security.
- Apply the memory-safety mindset to network parsers and protocol stacks.
- Understand the security implications of kernel-bypass networking (DPDK, XDP).

8.1 Network Packets at the Byte Level

Network security discussions often focus on high-level protocols—TLS, HTTP, OAuth. This chapter examines what happens at the raw byte level, where the abstraction gaps between protocol specifications and implementations create vulnerabilities.

An Ethernet frame arriving at a network interface is a sequence of bytes:

Listing 8.1: Ethernet frame (simplified)

Destination MAC (6) Source MAC (6) EtherType (2) Payload FCS (4)
--

Within the payload, an IPv4 packet:

Listing 8.2: IPv4 header (20 bytes)

Version (4b) IHL (4b) DSCP (6b) ECN (2b) Total Length (16b)
Identification (16b) Flags (3b) Fragment Offset (13b)
TTL (8b) Protocol (8b) Header Checksum (16b)
Source IP (32b)
Destination IP (32b)

Within the IP payload, a TCP segment:

Listing 8.3: TCP header (20 bytes)

Source Port (16b) Destination Port (16b)
Sequence Number (32b)
Acknowledgment Number (32b)

Data Offset (4b)	Reserved (6b)	Flags (6b)	Window (16b)
Checksum (16b)	Urgent Pointer (16b)		

These are the data structures that network parsers must process—and every parsing operation is a potential memory-safety bug.

8.2 Protocol Parsing Vulnerabilities

Every packet arriving at a host is parsed by the kernel’s network stack (or a kernel-bypass stack). Parsing is where memory-safety bugs live:

8.2.1 Length Field Validation

Protocol headers contain length fields that must be validated before use:

Listing 8.4: Vulnerable IP length parsing

```
int parse_ipv4(struct sk_buff *skb) {
    struct iphdr *ip = ip_hdr(skb);
    unsigned int hdr_len = ip->ihl * 4; // IHL is 4-bit header length
    unsigned int tot_len = ntohs(ip->tot_len);
    // BUG: if hdr_len < 20 or tot_len < hdr_len:
    // negative length, integer underflow, etc.
    if (skb->len < tot_len) return -EINVAL; // Good check
    // But what if ip->ihl < 5 (minimum)? Then hdr_len < 20.
    // What if ip->ihl > 15? Then hdr_len > 60 (max header).
    parse_options(skb, ip, hdr_len); // OOB read
}
```

The Linux kernel’s IP parser validates these fields correctly in the mainline, but hundreds of embedded TCP/IP stacks (in IoT devices, network cards, boot ROMs) may not. Off-by-one errors in length validation are a common source of CVEs.

8.2.2 TTL and Fragment Handling

IP fragmentation introduces state into what would otherwise be a stateless parser. The kernel must reassemble fragments, which requires:

- Tracking which fragments have arrived.
- Enforcing a maximum reassembly buffer size.
- Detecting overlapping fragments (which may be used to bypass firewalls).
- Timing out incomplete fragments.

Every aspect of this state machine has been vulnerable at some point. A classic attack is the *teardrop* attack: overlapping IP fragments cause a calculation error in fragment length, leading to a kernel panic.

8.3 Buffer Overflow in Network Parsers

Network parsers are buffer-overflow hotspots because they must handle attacker-controlled lengths:

Listing 8.5: Network buffer overflow pattern

```
int parse_protocol(const u8 *data, size_t len) {
    u8 option_buf[64];
    u16 opt_len = *(u16 *) (data + 2); // Attacker controls this!
    if (opt_len > 64) return -EINVAL; // Check? Maybe not.
}
```



```
memcpy(option_buf, data + 4, opt_len); // Overflow!
}
```

The Linux kernel handles tens of thousands of such length checks. Any one that is missing or incorrect is a potentially critical vulnerability.

8.4 Socket-Level Attacks

8.4.1 TCP Desynchronisation

An attacker who can observe or manipulate TCP traffic can desynchronise the TCP state machine by injecting packets with crafted sequence numbers. This can:

- Cause data corruption by injecting forged segments.
- Reset an established connection (RST attack).
- Blindly inject data into a TCP stream (if the sequence number is guessed or observed).

8.4.2 TCP Side Channels

The TCP header’s timestamp option (TSval/TSecr) can be used as a side channel to count packets or measure clock skew—useful for fingerprinting the sender’s operating system or network topology.

8.5 Kernel-Bypass Networking

Traditional networking goes through the kernel’s network stack, which provides safety (validation, isolation, scheduling) at the cost of performance. Kernel-bypass technologies (DPDK, XDP, AF_XDP) give user-space programs direct access to the network interface card (NIC).

Definition 8.1 (Kernel-Bypass). **Kernel-bypass networking** allows a user-space application to send and receive raw network frames without traversing the kernel’s network stack. The application communicates directly with the NIC hardware.

8.5.1 Security Implications

Kernel bypass trades safety for performance:

Aspect	Kernel networking	Kernel bypass
Packet validation	Kernel enforces checks	Application’s responsibility
Privilege separation	Kernel provides isolation	Application has direct HW access
Memory safety	Kernel validator has CFI	Application must validate
Mitigation access	ASLR, NX, SMEP/SMAP, KPTI	Same (kernel runs mitigations)
Bug impact	Kernel memory corruption	Still kernel context (via DMA)

A bug in a DPDK packet parser is particularly dangerous because:

1. The parser runs in user space but has access to DMA buffers—a bug gives direct hardware access.
2. There is no kernel validator between the NIC and the application to catch errors.

3. DPDK applications are typically written in C with extensive use of zero-copy (direct buffer manipulation), which amplifies the impact of memory-safety bugs.

8.5.2 eXpress Data Path (XDP)

XDP attaches a BPF program to the network interface, running before the kernel's network stack. This allows:

- Early packet filtering (faster than iptables/nftables).
- Packet modification (header rewrite, encapsulation).
- Redirection to user space via AF_XDP sockets.

XDP programs run in a BPF sandbox with verifier-enforced safety: bounded loops, no arbitrary memory writes, and termination guarantees. This makes XDP safer than DPDK for the same use case.

8.6 Wireless and Low-Level Attacks

Wireless networks add another layer of low-level complexity:

- **Frame injection:** crafting raw 802.11 frames to perform deauthentication attacks, beacon flooding, or evil-twin access points.
- **Wi-Fi parser bugs:** the 802.11 frame format includes numerous optional fields (HT/VHT/HE capabilities, information elements)—each is a potential parsing vulnerability in the firmware or kernel driver.
- **Firmware-level exploits:** Wi-Fi chipsets run proprietary firmware that processes raw frames. Bugs in this firmware can give an attacker arbitrary code execution on the Wi-Fi chip itself, often with DMA access to host memory.

Example 8.1 (Broadpwn (CVE-2017-9417)). A buffer overflow in Broadcom Wi-Fi firmware's handling of beacon frames allowed an attacker in range to execute code on the Wi-Fi chipset, which had full DMA access to the host's memory. No user interaction was required—only a nearby malicious access point.

8.7 Exercises

Exercise 8.1. *Given a raw Ethernet frame (as hex bytes), identify the source and destination MAC addresses, EtherType, and the IP version. Use a packet capture tool (tcpdump or Wireshark) to verify.*

Exercise 8.2. *An embedded TCP/IP stack parses an incoming TCP segment. The header length field (Data Offset) is 15, implying a 60-byte header. The total segment length is 40 bytes. What happens when the parser tries to read header options? What vulnerability class is this?*

Exercise 8.3. *Compare the security properties of XDP and DPDK for a packet-processing application. Under what circumstances would each be more secure?*

Exercise 8.4. *Research CVE-2020-11989 (a Linux kernel vulnerability in the netfilter subsystem). Describe the bug, its root cause, and how it might be exploited.*

Chapter 9

Cryptography for Systems Programmers

“Cryptography is not a source of security. Cryptography is a tool that amplifies security when used correctly, and amplifies disaster when misused.”

Learning Goals

- Distinguish between cryptographic primitives and understand their correct usage.
- Identify common low-level pitfalls in crypto code: side channels, RNG failures, padding oracles.
- Understand constant-time programming and why it matters.
- Explain how memory-safety bugs in crypto libraries subvert cryptographic guarantees.
- Apply the cryptographic engineer’s mindset to systems programming.

9.1 Cryptographic Primitives for Systems Programming

A systems programmer encounters three core primitive types:

9.1.1 Symmetric Encryption

A single key encrypts and decrypts data.

$$c = E(k, m) \quad m = D(k, c)$$

Common algorithms: AES (in various modes: AES-GCM, AES-CBC, AES-CTR), ChaCha20.

Key low-level details:

- AES operates on 16-byte blocks. A single-block encryption is 10–14 rounds of SubBytes, ShiftRows, MixColumns, AddRoundKey. Each round is a sequence of table lookups and XOR operations.
- AES-NI instructions (`aesenc`, `aesdec`, `aeskeygenassist`) perform these rounds in hardware, eliminating table-lookup side channels.
- Block cipher modes matter enormously: AES-ECB leaks plaintext patterns; AES-CBC requires unpredictable IVs; AES-GCM provides authenticated encryption.

9.1.2 Asymmetric (Public-Key) Encryption

Two keys: public for encryption, private for decryption.

$$c = E(pk, m) \quad m = D(sk, c)$$

Common algorithms: RSA, ECDH, Ed25519.

Key low-level details:

- RSA operates on integers modulo a large composite $N = pq$. Encryption is $c = m^e \bmod N$. Decryption is $m = c^d \bmod N$. The exponentiation is implemented with square-and-multiply, which is vulnerable to timing side channels if not implemented in constant time.
- Elliptic curve operations are point additions and scalar multiplications on elliptic curves over finite fields. The security depends on the hardness of the discrete logarithm problem. Incorrect implementations can compute the wrong curve or fail to validate points.

9.1.3 Hash Functions

A deterministic, one-way function with collision resistance.

$$h = H(m)$$

Common algorithms: SHA-256, SHA-3, BLAKE2.

Key low-level details:

- SHA-256 processes 64-byte blocks through 64 rounds of compression. The state is 8 32-bit words. Like AES, implementations that use table lookups for the compression function may leak state through cache timing.
- Length-extension attacks: SHA-256 (and all Merkle-Damgård hashes) allow computing $H(m \parallel \text{pad}(m) \parallel \text{extension})$ given only $H(m)$ and $\text{len}(m)$, without knowing m . This is why HMAC exists.

9.2 Common Low-Level Crypto Pitfalls

9.2.1 Constant-Time Programming

The most important concept in cryptographic engineering: cryptographic code must take the same amount of time regardless of its inputs.

Definition 9.1 (Constant-Time Code). **Constant-time** code executes in a time that depends only on the length of the inputs, not on their values. Any data-dependent timing variation can be used as a side channel to leak secret information.

Consider a naive string comparison:

Listing 9.1: Variable-time comparison

```
int memcmp_vartime(const void *a, const void *b, size_t n) {
    const unsigned char *pa = a, *pb = b;
    for (size_t i = 0; i < n; i++) {
        if (pa[i] != pb[i])
            return 0; // Returns early on first mismatch!
    }
    return 1;
}
```

This function returns as soon as it finds a mismatch. An attacker who can measure the response time can determine, byte by byte, where the correct key differs from their guess. This is a *timing oracle*.

The constant-time version:

Listing 9.2: Constant-time comparison

```
int memcmp_consttime(const void *a, const void *b, size_t n) {
    const unsigned char *pa = a, *pb = b;
    unsigned char result = 0;
    for (size_t i = 0; i < n; i++) {
        result |= pa[i] ^ pb[i]; // Always processes all bytes
    }
    return (result == 0);
}
```

Principle 9.1 (No Secret-Dependent Branches or Lookups). To write constant-time code: (1) never branch based on secret data, (2) never index memory with secret data (cache side channel), (3) always process all elements.

9.2.2 Random Number Generation

Cryptography requires unpredictable random numbers. The standard C `rand()` is not suitable. The Linux kernel exposes random bytes through `/dev/urandom` (or the `getrandom` syscall).

Listing 9.3: Reading random bytes on Linux

```
#include <sys/random.h>

void generate_key(uint8_t *key, size_t len) {
    ssize_t n = getrandom(key, len, 0);
    if (n != (ssize_t)len) {
        // Handle error (should not happen with GRND_NONBLOCK)
    }
}
```

Common RNG mistakes:

- Seeding with `time(NULL)`: if the attacker knows when the seed was generated, they can reproduce the entire random sequence.
- Using `rand()` or `random()`: these are pseudo-random and not cryptographically secure.
- Reading from `/dev/random` instead of `/dev/urandom`: `/dev/random` blocks when the kernel entropy pool is low, causing denial-of-service.
- Failing to check the return value of `read/getrandom`.

9.2.3 Padding Oracle Attacks

A padding oracle attack exploits the fact that decryption failures reveal whether padding was valid:

Listing 9.4: AES-CBC padding oracle

```
int decrypt_and_verify(const uint8_t *ciphertext, size_t len) {
    uint8_t plaintext[MAX_LEN];
    int pt_len = aes_cbc_decrypt(ciphertext, len, key, iv, plaintext);
    if (pt_len < 0)
        return -1; // Bad padding: attacker learns this!
    if (!verify_mac(plaintext, pt_len))
        return -2; // Bad MAC
}
```

```

    return pt_len;
}

```

An attacker who can distinguish “bad padding” from “bad MAC” can decrypt arbitrary ciphertexts byte by byte. The fix: always validate MAC first, and return a single error code regardless of what failed.

9.3 Memory Safety in Cryptographic Code

Cryptographic code is particularly sensitive to memory-safety bugs because the security guarantees collapse entirely if an attacker can read or write arbitrary memory.

9.3.1 Secret Management

- **Secrets on the heap:** a secret stored on the heap may survive in freed memory, then be reused by a different allocation.
- **Secrets in core dumps:** a core dump captures all process memory, including private keys.
- **Secrets in swap:** swapped-out pages containing secrets are written to disk.
- **Secrets in side channels:** cache timing, branch prediction, and power analysis can leak secret-dependent access patterns.

Listing 9.5: Secure memory zeroing

```

void secure_zero(void *buf, size_t len) {
    volatile unsigned char *p = buf;
    for (size_t i = 0; i < len; i++)
        p[i] = 0;
    // 'volatile' prevents the compiler from optimising away the zeroing
}

```

Using `memset(buf, 0, len)` is not secure for clearing secrets: the compiler may optimise it away since the buffer is not read after being written. The `volatile` qualifier forces the zeroing to be emitted.

9.3.2 Library-Level Vulnerabilities

The most devastating crypto vulnerabilities often come from memory-safety bugs in the library itself rather than cryptographic weaknesses:

- **Heartbleed (CVE-2014-0160):** a missing bounds check in OpenSSL’s heartbeat extension allowed reading up to 64 KB of arbitrary process memory—including private keys.
- **CCA_Decrypt_OpenSSL:** a use-after-free in OpenSSL’s X.509 certificate parsing could lead to code execution when processing a malicious certificate.
- **bleichenbacher oracle:** an implementation bug in RSA PKCS#1 v1.5 decryption (not a bounds check but a timing variation) allowed breaking RSA encryption.

Principle 9.2 (Crypto Libraries Are Not Immune to Memory Bugs). A cryptographic algorithm may be mathematically unbreakable, but its implementation in C is subject to the same memory-safety bugs as any other C program. The security of a cryptosystem is the security of its weakest component.

9.4 Authenticated Encryption

The modern approach to symmetric encryption is *authenticated encryption*: encryption that guarantees both confidentiality and integrity. AES-GCM and ChaCha20-Poly1305 are the dom-

inant AEAD (Authenticated Encryption with Associated Data) constructions.

$$(c, t) = \text{AEAD_Enc}(k, \text{nonce}, ad, \text{plaintext})$$

Where t is an authentication tag. On decryption, the tag is verified before the plaintext is released:

Listing 9.6: AES-GCM decryption must verify tag

```
int aead_decrypt(const uint8_t *ciphertext, size_t len,
                 const uint8_t *tag, const uint8_t *key,
                 const uint8_t *nonce, const uint8_t *ad,
                 size_t ad_len, uint8_t *plaintext) {
    // 1. Decrypt and verify tag atomically
    if (crypto_aead_decrypt(plaintext, NULL, NULL,
                           ciphertext, len, ad, ad_len, tag, key, nonce) != 0)
        return -1; // Authentication failure
    // 2. Only now is plaintext safe to use
    return len;
}
```

Never decrypt without verifying the tag. Never release decrypted plaintext before tag verification.

9.5 Exercises

Exercise 9.1. *Given the following constant-time vulnerabilities, identify the leak:*

```
int ct_compare(const uint8_t *a, const uint8_t *b, size_t n) {
    volatile uint8_t diff = 0;
    for (size_t i = 0; i < n; i++) {
        diff |= a[i] ^ b[i];
    }
    return diff;
}
```

Is this truly constant-time? What if `a` or `b` is paged out?

Exercise 9.2. *Design a constant-time function that selects one of two buffers based on a secret condition: `void ct_select(int condition, uint8_t *out, const uint8_t *a, const uint8_t *b, size_t n)`. If condition is true, copy `a` to `out`; otherwise copy `b`. Both branches must execute regardless of condition.*

Exercise 9.3. *Why is AES-GCM preferred over AES-CBC + HMAC? What security property does GCM provide that CBC+HMAC requires extra care to achieve?*

Exercise 9.4. *Research the Debian OpenSSL RNG vulnerability (CVE-2008-0166). What was the root cause? How many cryptographic keys were affected? What lesson does this teach about RNG in systems programming?*

Chapter 10

Secure Development Practices

“Security is not a feature. Security is a property of the engineering process.”

Learning Goals

- Integrate security into the software development lifecycle for systems programming.
- Apply threat modelling to low-level systems (protocol parsers, device drivers, kernels).
- Use static and dynamic analysis tools effectively.
- Write and enforce secure coding standards for C and C++.
- Design test harnesses that catch memory-safety bugs before release.

10.1 The Secure Development Lifecycle (SDL)

For low-level software, security cannot be retrofitted. It must be embedded in each phase:

Phase	Security activity	Low-level focus
Requirements	Threat modelling	Identify trust boundaries, attacker model
Design	Security architecture	Privilege separation, minimal API surface
Implementation	Secure coding standards	Bounds checking, safe API usage, format strings
Testing	Fuzzing + sanitizers	Coverage-guided fuzzing with ASan/UBSan
Audit	Code review + static analysis	Manual review of memory-unsafe patterns
Release	Binary hardening	Compiler flags, RELRO, PIE, seccomp
Response	Incident plan	Patch process for memory-corruption CVEs

10.2 Threat Modelling for Low-Level Systems

Threat modelling asks: what can an attacker control, and what can they achieve?

10.2.1 Identifying Trust Boundaries

For a system component, ask:

- Where does the input come from? (network, file, user, another process, kernel)
- Is the input attacker-controlled? (yes for network parsers, maybe for file parsers, no for local configuration)
- What happens to the input? (parsed, copied, stored, executed)
- What privileges does this component have? (user, root, kernel)
- What happens if the component crashes? (DoS, privilege escalation, information leak)

Definition 10.1 (Trust Boundary). A **trust boundary** is the point at which data crosses from a less-trusted to a more-trusted environment. Every crossing is a place where validation must occur.

10.2.2 The STRIDE Model

Microsoft’s STRIDE threat classification helps identify threat types:

Threat	Definition	Low-level example
Spoofing	Pretending to be someone else	Forged IP source address
Tampering	Modifying data or code	Buffer overflow overwriting return address
Repudiation	Denying an action	No audit trail memory corruption
Information disclosure	Leaking data	Format string leaking stack
Denial of service	Making system unavailable	Infinite loop in a protocol parser
Elevation of privilege	Getting more access	Kernel exploit via ioctl

10.3 Coding Standards for Memory Safety

10.3.1 Rule 1: Always Bounds-Check

Every array access, every pointer arithmetic operation, every copy with a programmer-specified length must have a corresponding bounds check.

Listing 10.1: Bounds checking pattern

```
int process_message(const uint8_t *buf, size_t buf_len) {
    if (buf_len < HEADER_SIZE)
        return -EINVAL;
    uint16_t payload_len = ntohs(*(uint16_t *) (buf + 2));
    if (payload_len > buf_len - HEADER_SIZE) // CRITICAL: bounds check
        return -EINVAL;
    // Safe to copy payload_len bytes
    memcpy(payload_buf, buf + HEADER_SIZE, payload_len);
    return 0;
}
```

10.3.2 Rule 2: Use Safe String Functions

Avoid	Use instead	Reason
<code>strcpy</code>	<code>strncpy, snprintf</code>	Always specify destination size
<code>strcat</code>	<code>strlcat</code>	Same
<code>sprintf</code>	<code>snprintf</code>	No buffer size argument
<code>gets</code>	<code>fgets</code>	No bounds at all
<code>scanf("%s", buf)</code>	<code>scanf("%Ns", buf)</code>	Specify max field width

10.3.3 Rule 3: Validate All Integer Arithmetic

Integer overflows can mask buffer overflows:

Listing 10.2: Safe integer arithmetic

```
#include <stdckdint.h> // C23

bool safe_allocation(size_t count, size_t elem_size, void **out) {
    size_t total;
    if (ckd_mul(&total, count, elem_size)) // Checked multiplication
        return false; // Overflow
    *out = malloc(total);
    return *out != NULL;
}
```

The `ckd_*` functions (checked integer arithmetic) return true on overflow. Use them for every multiplication or addition that involves a length derived from external input.

10.3.4 Rule 4: Avoid Variable-Length Arrays (VLAs)

VLAs on the stack are dangerous because the size comes from a variable—if the variable is attacker-controlled, the stack may overflow:

Listing 10.3: Never use VLAs with untrusted sizes

```
void process(size_t size) {
    // DANGEROUS: if size is large, this overflows the stack
    // and size is attacker-controlled
    char buf[size];
}
```

10.3.5 Rule 5: Initialize All Memory

Uninitialised memory may contain stale data from a previous allocation—including secrets or function pointers:

Listing 10.4: Zero-initialise allocated memory

```
struct config *cfg = calloc(1, sizeof(*cfg)); // calloc zeros memory
// NOT: struct config *cfg = malloc(sizeof(*cfg)); // uninitialised!
```

10.4 Fuzzing for Security Bugs

Fuzzing is the single most effective technique for finding memory-safety bugs. Coverage-guided fuzzers (libFuzzer, AFL++) instrument the target binary and mutate inputs to maximise code coverage.

Listing 10.5: libFuzzer harness for a parser

```
#include <stdint.h>
#include <stddef.h>

int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    // Parse the input
    struct message msg;
    if (parse_message(data, size, &msg) < 0)
        return 0; // Valid input rejected (not a bug)
    // Trigger deeper processing
    process_message(&msg);
    return 0;
}
```

Run with sanitizers:

Listing 10.6: Running the fuzzer with sanitizers

```
$ clang -fsanitize=address,fuzzer -g -O1 fuzz_harness.c parser.c -o fuzz_parser
$ ./fuzz_parser -max_len=4096 -rss_limit_mb=2048 corpus/
```

Sanitizer integration is critical:

- **AddressSanitizer:** detects heap/stack/global overflows, use-after-free, double free.
- **UndefinedBehaviorSanitizer:** detects integer overflow, misaligned access, null pointer dereference.
- **MemorySanitizer:** detects reads of uninitialised memory (requires all dependencies to be instrumented).
- **LeakSanitizer:** detects memory leaks.

10.5 Static Analysis

Static analysis tools find certain classes of bugs without running the code:

- **Clang Static Analyzer:** path-sensitive analysis for null dereference, use-after-free, memory leaks.
- **Infer (from Facebook):** compositional analysis for large codebases.
- **CodeQL:** query-based analysis that can express complex security patterns (e.g., “all format strings where the format argument is user-controlled”).
- **cppcheck:** simple pattern-based checker for common errors.

Example 10.1 (Grep-Based Security Review). Before automated analysis, review for dangerous patterns:

Listing 10.7: Grep patterns for security review

```
# Find unbounded string copies
$ grep -rn 'strcpy\|strcat\|sprintf\|gets' src/

# Find unsafe format strings
$ grep -rn 'printf(user\|fprintf(user\|syslog(user' src/

# Find integer casts that might truncate
```

```
$ grep -rn '(int)\|(uint16_t)\|(uint8_t)' src/ | grep -v 'hton\|ntoh'
```

10.6 Compiler Hardening for Release Builds

For production builds, enable all hardening flags:

Listing 10.8: Recommended compiler flags for security

```
CFLAGS = -fstack-protector-strong \
         -fPIE -pie \
         -D_FORTIFY_SOURCE=3 \
         -Wl,-z,relro,-z,now \
         -Wl,-z,noexecstack \
         -fcf-protection=full \
         -mbranch-protection=standard
```

10.7 Code Review Checklist for Low-Level Security

A code review checklist for memory-safety-critical code:

1. Every `memcpy`, `memmove`, `strcpy`, `snprintf` has a validated length argument.
2. Every array index derived from external input is bounds-checked.
3. Every integer arithmetic operation on lengths uses checked arithmetic (`ckd_*` or manual overflow checks).
4. Format strings are never user-controlled.
5. All `switch` statements on external input have a `default` case.
6. All structs copied from/to user space use `copy_from_user` / `copy_to_user` (kernel) or validate sizes (user).
7. No variable-length arrays on the stack with attacker-controlled sizes.
8. All memory containing secrets is explicitly zeroed with `volatile` or `explicit_bzero`.
9. Every allocation from `malloc` is checked for `NULL`.
10. Function pointers are validated before calling (if feasible).
11. All file descriptors are checked for validity before use.
12. Every error path properly cleans up (frees memory, closes FDs, releases locks).

10.8 Building a Security Culture

Technical measures are necessary but insufficient. The team must:

- **Prioritise memory safety:** treat a memory-safety bug with the same severity as a production outage.
- **Invest in testing infrastructure:** continuous fuzzing with corpus management.
- **Conduct regular security reviews:** quarterly reviews of new code with the checklist above.
- **Adopt memory-safe components:** when feasible, use Rust or other safe languages for new protocol parsers, or isolate unsafe code behind safe APIs.
- **Track metrics:** number of memory-safety bugs found per release, time to fix, fuzzing coverage percentage.

Principle 10.1 (The Ultimate Mitigation Is Process). Every mitigation—canaries, ASLR, CFI, seccomp—can be bypassed given enough skill and time. The only defence that scales is preventing exploitation from being necessary: find and fix bugs before they ship.

10.9 Exercises

Exercise 10.1. Apply the *STRIDE* threat model to a USB device driver in the Linux kernel. List at least one threat per category. What trust boundaries exist between user space and the driver?

Exercise 10.2. Write a *libFuzzer* harness for a function that parses a simple binary protocol (e.g., a fixed header followed by a variable-length payload). Compile it with *AddressSanitizer* and run it for 1 million iterations. Report any crashes.

Exercise 10.3. Audit the following code for security bugs. List every violation of the review checklist:

```
int process_config(const char *input, size_t len) {
    char name[64];
    char *value = malloc(len);
    int n = sscanf(input, "%s=%s", name, value);
    if (n != 2) return -1;
    set_config(name, value);
    free(value);
    return 0;
}
```

Exercise 10.4. Research the Chromium sandbox architecture. How does it use privilege separation to contain memory-safety bugs in the rendering engine? What system calls are allowed in the sandboxed process?

Bibliography

- [1] Aleph One. Smashing the stack for fun and profit, 1996. URL <http://phrack.org/issues/49/14.html>.
- [2] scut and team teso. Exploiting format string vulnerabilities, 2003. URL <https://crypto.stanford.edu/cs155/papers/formatstring-1.2.pdf>.
- [3] Martin Abadi, Mihai Budiu, Úlfar Erlingsson, and Jay Ligatti. Control-flow integrity: Principles, implementations, and applications. In *ACM CCS*, pages 340–353, 2005.
- [4] Hovav Shacham. The geometry of innocent flesh on the bone: Return-into-libc without function calls (on the x86). pages 552–561, 2007.
- [5] László Szekeres, Mathias Payer, Tao Wei, and Dawn Song. Eternal war in memory. pages 214–228, 2013.
- [6] Andrea Bittau, Adam Belay, Ali Mashtizadeh, David Mazières, and Dan Boneh. Hacking blind. In *IEEE S&P*, pages 213–227, 2014.